



Entrega 5 - Resultados de la Experimentación

GRUPO 16

SERGIO FERNANDO BARRERA MOLANO

202517034

HAROLD ANDRÉS BARTOLO MOSCOSO

202513889

JUAN JOSÉ RESTREPO BONILLA

202516633



Resultados de la experimentación - Hogar de los Alpes

Este documento presenta los resultados cuantitativos y cualitativos de los experimentos ejecutados sobre la POC (4 microservicios + orquestador de sagas + BFF + Apache Pulsar) para validar los tres escenarios de calidad escogidos en la Entrega 3, uno por cada atributo priorizado en la Entrega 2, y concluye si la hipótesis de cada experimento se cumplió.

1. Marco de la experimentación

Elemento	Definición
Sistema bajo prueba	Transacción larga aceptar cotización (cotizaciones → pagos → trabajos, orquestada por el servicio saga), con notificaciones como consumidor y el BFF como puerta de entrada
Atributos (Entrega 2)	Escalabilidad · Modificabilidad · Disponibilidad
Escenarios (Entrega 3)	E1 pico climático 4x · E6 evolución del contrato v1→v2 · E7 caída de la pasarela de pagos
Entornos	Modo B (los 6 procesos locales, adaptador de broker por archivos) para las mediciones repetibles de este documento; Modo A (Docker + Apache Pulsar 3.2.4) para validar el comportamiento contra el broker real
Herramientas	Scripts escenarios/*.py (publican comandos, consultan por HTTP, verifican con aserciones), Saga Log (SQL), Postman/newman (28 requests, 69 aserciones)
Hardware de referencia	1 proceso por servicio, sin réplicas, SQLite por servicio; contenedor Linux de 2 vCPU

Tabla 1. Marco de la experimentación.

Nota metodológica. Los valores absolutos (milisegundos, transacciones/s) corresponden a la POC sin réplicas y no son los de producción; lo que se valida es el comportamiento frente al estímulo (rechazos, pérdidas, duplicados, roturas, disponibilidad) y la forma en que escalan las métricas. Cada corrida se repitió al menos 3 veces; se reportan valores representativos.

2. E1: Escalabilidad: pico de creación masiva

Escenario (Entrega 3). Una granizada multiplica por 4 las solicitudes de los partners (~290 → ~1.160 req/s). Medida: 0 % rechazos; ACK p99 < 800 ms; la cola drena a estado estable en < 6 h.

Hipótesis H1. Con ingestión asíncrona por comandos (buffering en el broker) y consumidores idempotentes, el sistema absorbe una ráfaga N veces mayor que su capacidad de procesamiento sin rechazar ninguna solicitud, y



drena la cola con conteos exactos (ni faltantes ni duplicados), con un tiempo de drenaje que crece linealmente con N.

Diseño. Ráfaga de N comandos `CrearCotizacion`; por cada `CotizacionCreada` se inicia la transacción larga (`IniciarSagaAceptacion`); se mide (a) rechazos y throughput de publicación, (b) tiempo hasta que pagos alcanza exactamente N reservas, (c) sagas completadas.

Resultados cuantitativos

N (ráfaga)	Publicación	Rechazos	Drenaje end-to-end	Transacciones/s (drenaje)	Reservas	Sagas COMPLETADAS
30	0,00 s (28.663 cmd/s)	0	1,0 s	30	30/30	30/30
300	0,01 s (47.944 cmd/s)	0	8,1 s	37	300/300	300/300
1.000	0,02 s (47.969 cmd/s)	0	26,9 s	37	1.000/1.000	1.000/1.000

Tabla 2. Resultados de E1 para tres tamaños de ráfaga.

Duración de las sagas individuales bajo la ráfaga de 1.000 (Saga Log, `fecha_fin` - `fecha_inicio`): mínimo 0,75 s · mediana 13,0 s · p95 17,3 s · máximo 17,4 s. Entradas de Saga Log generadas: 15.180 (11 por transacción exitosa).

Análisis

- **Absorción.** La publicación es ~1.300 veces más rápida que el drenaje (48.000 cmd/s vs 37 tx/s): el broker absorbe íntegramente el pico y ninguna solicitud se rechaza. Es exactamente el mecanismo que protege al negocio en la granizada: aceptar ahora, procesar a ritmo sostenible.
- **Linealidad.** El tiempo de drenaje crece linealmente con N (8,1 s → 26,9 s para 300 → 1.000, ×3,3 para ×3,3) y el throughput de drenaje se estabiliza en ~37 tx/s con un solo proceso por servicio. La táctica de la Entrega 3 (autoescalar consumidores por lag de suscripción) se apoya en que las suscripciones son Shared y los consumidores idempotentes: añadir réplicas reparte el backlog sin cambiar código.
- **Exactitud.** En las tres cargas, reservas y sagas completadas son exactamente N: la idempotencia (marca del mensaje en la misma transacción que el efecto) elimina duplicados aunque el broker re-entregue.



- **Latencia percibida.** La mediana de 13 s por saga bajo la ráfaga de 1.000 es el precio de la cola: el ACK al cliente (202) es inmediato, pero el agendamiento tarda. Coincide con el tradeoff declarado en la Entrega 2 (§1.2: se sacrifica latencia de operación, no de sistema).

Conclusión H1: se cumple. 0 rechazos, conteos exactos y crecimiento lineal en las tres cargas. Extrapolación con reservas: para sostener 1.160 req/s del escenario real harían falta ~32 réplicas de la cadena a la tasa medida (37 tx/s por réplica), lo que es alcanzable con particiones por `id_trabajo` y autoescalado; la latencia por saga en pico exige dimensionar el backlog para que el drenaje termine dentro de las 6 h del escenario (a 37 tx/s, 1.000 tx drenan en 27 s; el pico de 48 h del escenario requiere las réplicas indicadas).

3. E6: Modificabilidad: evolución del contrato v1 → v2

Escenario (Entrega 3). Gestión de Trabajos/Cotizaciones publica una versión nueva de un evento con 8+ consumidores en producción (30+ partners). Medida: downtime 0; consumidores rotos 0; convivencia v1/v2 ≥ 30 días.

Hipótesis H6. Con eventos de integración versionados en el envelope (`specversion`) y una política BACKWARD (una versión nueva solo agrega campos opcionales), un productor puede publicar v2 mientras consumidores escritos contra v1 siguen procesando v1 y v2 sin cambios ni reinicio.

Diseño. Cotizaciones publica `CotizacionAceptada` v2 (nuevo campo `pais`, necesario para la expansión global). Notificaciones es un consumidor escrito contra v1. Se publican al mismo tópico un v1 y un v2 y se verifica que el consumidor procesa ambos y registra la versión recibida.

Resultados

Métrica	Resultado
Consumidores rotos	0 (notificaciones procesó v1 y v2; log: <code>CotizacionAceptada</code> v2 recibida - campo <code>pais</code> ignorado (consumidor v1))
Convivencia v1/v2 en el mismo tópico	Sí (<code>versiones registradas = ['v1', 'v2']</code>)
Downtime del productor o consumidores	0 (ningún reinicio)
Cambios de código en consumidores para tolerar v2	0 (acceso por <code>data.get('pais', 'CO')</code>)
Cambios de código para que la saga y pagos usaran <code>pais</code> (consumidores v2)	1 línea por consumidor: leer el campo nuevo



Métrica	Resultado
Versiones distintas de un mismo tipo de evento en un event store	El agregado Cotizacion guarda CotizacionCreada v1 y CotizacionAceptada v2 en la misma historia y se reconstruye por replay sin conversión

Tabla 3. Resultados de E6.

Análisis cualitativo

- La decisión de versionar por mensaje y no por nombre de tópico (**eventos-cotizacion-v2**) es lo que permite la convivencia: cada consumidor migra a su ritmo, requisito con 30+ partners que no despliegan sincronizados.
- El costo de la política BACKWARD es que nunca se puede renombrar ni eliminar un campo; un cambio incompatible exige un tipo de evento nuevo. Lo aceptamos: el contrato es la interfaz pública del sistema y su estabilidad vale más que la libertad del productor.
- Durante la Entrega 5 el contrato volvió a evolucionar (nuevos eventos **CotizacionRevertida**, **PagoRevertido**, **TrabajoRechazado**; nuevos comandos de compensación) y ningún consumidor existente se rompió: los consumidores ignoran tipos que no conocen. Es evidencia adicional, no planeada, de la misma propiedad.
- Lo que la POC no valida:** la duración real de la convivencia (≥ 30 días) ni un registro de esquemas que rechace automáticamente cambios incompatibles (Avro + Schema Registry lo daría; es un adaptador adicional).

Conclusión H6: se cumple. El sistema acepta una nueva versión del contrato sin downtime ni consumidores rotos, con v1 y v2 conviviendo en el mismo tópico.

4. E7: Disponibilidad: caída de la pasarela de pagos (escenario crítico)

Escenario (Entrega 3). La pasarela de pagos responde 5xx durante 30 minutos en hora pico. Medida: el núcleo (crear/cotizar/aceptar) sigue al 100 %; 0 pagos perdidos; 0 duplicados; recuperación < 5 min tras volver.

Hipótesis H7. Si el cobro se desacopla del núcleo mediante comandos por broker y el consumidor es idempotente, la caída del servicio de pagos no afecta la disponibilidad del núcleo, no pierde ninguna transacción (RPO = 0) y, al recuperarse, todas las transacciones pendientes terminan sin duplicados.

Diseño. Se detiene el proceso/contenedor de pagos; con pagos caído se crean y se inician N transacciones largas; se sondea la disponibilidad del núcleo; se reanuda pagos; se mide el drenaje hasta trabajos y las sagas



completadas; finalmente se re-inyectan 5 eventos ya procesados (simulando la re-entrega at-least-once del broker) y se cuentan duplicados.

Resultados cuantitativos

Métrica	N = 15	N = 50
Disponibilidad del núcleo durante la caída (sondas HTTP)	100 % (10/10)	100 % (10/10)
Transacciones iniciadas con pagos caído	15/15	50/50
Comandos <code>RetenerPago</code> retenidos en el broker	15	50
Pagos perdidos tras la recuperación (RPO)	0	0
RTO (rearranque → cola drenada)	0,5 s	1,1 s
Cadena cerrada hasta trabajos	15/15	50/50
Sagas COMPLETADAS en el Saga Log	15/15	50/50
Duplicados con 5 re-entregas inyectadas	0	0
Veredicto del script	CUMPLIDO	CUMPLIDO

Tabla 4. Resultados de E7 para N = 15 y N = 50.

En Modo A (Docker + Pulsar) el mismo escenario (`escenarios/e7_docker.sh`) reproduce el comportamiento: durante `docker compose stop pagos`, `GET /bff/health` responde 503 con `"pagos": "down"` mientras `GET /bff/cotizaciones` sigue en 200.

Análisis

- **El núcleo no se entera.** Las sondas a cotizaciones no fallan porque pagos no es una dependencia síncrona de nadie: el orquestador publica `RetenerPago` a `comandos-pago` y ese comando espera en el broker. Las 50 sagas quedan en `EN_CURSO` · `paso RETENER_PAGO`, visibles en el Saga Log, que es exactamente la información que soporte necesitaría durante un incidente real (`SELECT ... FROM sagas WHERE estado='EN_CURSO' AND paso_actual='RETENER_PAGO'`).
- **RPO = 0 por diseño.** El broker retiene hasta el `ack`, y el `ack` ocurre después de aplicar el efecto (nunca antes). El RTO medido (1,1 s para 50) es el tiempo de drenar el backlog; crece con el volumen retenido, por eso el escenario real fija < 5 min y exige dimensionar consumidores.
- **Idempotencia probada, no supuesta.** Las 5 re-entregas fueron rechazadas por la tabla `eventos_procesados` (log: `duplicado ... IGNORADO (idempotencia) ×5`), y las sagas no avanzaron dos veces.



- **Con orquestación la propiedad se mantiene** (Entrega 4 la probó con coreografía): la saga espera el evento de pagos el tiempo que haga falta; su estado está en la base, no en memoria.

Conclusión H7: se cumple. Disponibilidad del núcleo 100 %, RPO = 0, RTO acotado por el backlog, 0 duplicados, transacciones completadas 50/50.

5. La saga como experimento adicional (Entrega 5)

Aunque no es uno de los tres escenarios, la transacción larga se sometió a dos pruebas que aportan evidencia cualitativa sobre **consistencia**, el atributo que la Entrega 2 decidió sacrificar en su forma inmediata a cambio de disponibilidad y escalabilidad:

Caso	Resultado	Evidencia
Transacción exitosa	Saga COMPLETADA en 11 entradas de log; cotización ACEPTADA, pago RETENIDO, trabajo AGENDADO	Postman carpeta 1 (7 requests, todas verdes)
Fallo en el paso 3 (proveedor sin disponibilidad)	Saga COMPENSADA en 17 entradas; compensaciones en orden inverso (RevertirPago → RevertirAceptacion); estado final consistente: cotización EMITIDA (v3), pago LIBERADO, trabajo RECHAZADO	Postman carpeta 2 (6 requests), <code>probar_saga.sh</code> , consultas SQL de <code>docs/SAGA.md</code>
Duración de una transacción sin carga	0,75 s de INICIO a FIN (mínimo observado)	Saga Log

Tabla 5. Pruebas de la transacción larga.

La conclusión cualitativa: la consistencia eventual que el sistema ofrece es **gobernada**, hay un registro de en qué estado quedó cada transacción y por qué, lo que la hace aceptable para el negocio (soporte puede explicar cada caso a un partner).

6. Conclusiones generales

1. **Las tres hipótesis se cumplieron.** El sistema absorbe picos sin rechazar (H1), evoluciona sus contratos sin romper consumidores (H6) y mantiene el núcleo disponible ante la caída de una dependencia crítica sin perder ni duplicar transacciones (H7).
2. **Los resultados son coherentes con las decisiones de las entregas anteriores.** Los tres atributos priorizados en la Entrega 2 se comportan como el árbol de utilidad anticipó, y los atributos que se sacrificaron



(latencia de operación, consistencia inmediata, simplicidad operativa) aparecen como costos medibles: 13 s de mediana por saga bajo ráfaga, estados intermedios visibles en el Saga Log, seis contenedores que operar.

3. **La arquitectura escala por adición, no por rediseño.** El throughput de drenaje es lineal y constante por réplica; crecer a los volúmenes de la expansión global (36.000 trabajos/día, picos 4x) es una decisión de capacidad (réplicas, particiones, clúster Pulsar), no de código.
4. **Lo que la POC no prueba y queda como trabajo futuro:** la pérdida de una zona del broker (E8, requiere clúster Pulsar multi-broker), el rate limiting por partner (E9), la duración real de convivencia de versiones, y las cifras absolutas de producción. Los refinamientos de la arquitectura derivados de esta experimentación están en `docs/REFINAMIENTO-ARQUITECTURA.md`.

Anexo A. Cómo reproducir

```
# Modo B (sin Docker): todas las mediciones de este documento
bash escenarios/validar_todo.sh                # E1 (N=30), E6, E7 (N=15)
N=1000 python escenarios/escenario_e1_escalabilidad.py
N_CAIDA=50 python escenarios/escenario_e7_disponibilidad.py
bash escenarios/probar_saga.sh                  # saga exitosa + compensada
sqlite3 servicios/saga/src/saga/api/saga.db "SELECT estado, COUNT(*) FROM sagas GROUP BY estado;"

# Modo A (Docker + Pulsar)
docker compose run --rm -e N=1000 escenarios python escenario_e1_escalabilidad.py
bash escenarios/e7_docker.sh
```

Anexo B. Salidas crudas de las corridas reportadas

```
== E1 N=30:      publicados 30 en 0.00s (28,663 comandos/s) – 0 rechazos · drenaje 30 reservas en 1.0s ·
CUMPLIDO
== E1 N=300:    publicados 300 en 0.01s (47,944 comandos/s) – 0 rechazos · drenaje 300 reservas en 8.1s
· CUMPLIDO
== E1 N=1000:   publicados 1000 en 0.02s (47,969 comandos/s) – 0 rechazos · drenaje 1000 reservas en
26.9s · CUMPLIDO
== E6:          consumidor v1 procesó ambas: versiones nuevas registradas = ['v1', 'v2'] · CUMPLIDO
== E7 N=50:     disponibilidad 100% (10/10 sondas) · RTO = 1.1s · reservas 50/50 · perdidos = 0 ·
trabajos 50/50 · duplicados 0 con 5 re-entregas · sagas COMPLETADAS 50/50 · CUMPLIDO
Saga Log:       1380 sagas COMPLETADAS · duración ms: min 754 · mediana 13011 · p95 17322 · max 17446 ·
15180 entradas
```



© - **Derechos Reservados:** la presente obra, y en general todos sus contenidos, se encuentran protegidos por las normas internacionales y nacionales vigentes sobre propiedad Intelectual, por lo tanto su utilización parcial o total, reproducción, comunicación pública, transformación, distribución, alquiler, préstamo público e importación, total o parcial, en todo o en parte, en formato impreso o digital y en cualquier formato conocido o por conocer, se encuentran prohibidos, y solo serán lícitos en la medida en que se cuente con la autorización previa y expresa por escrito de la Universidad de los Andes.

De igual manera, la utilización de la imagen de las personas, docentes o estudiantes, sin su previa autorización está expresamente prohibida. En caso de incumplirse con lo mencionado, se procederá de conformidad con los reglamentos y políticas de la universidad, sin perjuicio de las demás acciones legales aplicables.